

Distributed Sparse Matrix–Vector Multiplication (SpMV) with MPI cuda-aware

Titien MARIE
Erasmus Student
University of Trento, Italy
 titienjacques.marie@studenti.unitn.it

I. INTRODUCTION

Sparse Matrix-Vector Multiplication (SpMV) is one of the most common operations in scientific computing, appearing in iterative solvers, graph algorithms and machine learning. As matrix sizes grow beyond the memory capacity of a single GPU, distributing the computation across multiple GPUs becomes necessary.

Building on our previous single-GPU SpMV implementation ([1]), this work investigates how to scale SpMV across up to 4 NVIDIA A30 GPUs using MPI and CUDA. We distribute the matrix rows across GPUs, exchange the required vector entries between ranks via CUDA-aware MPI, and perform the local computation using our CSR warp kernel with shared memory. We evaluate our implementation on 10 real sparse matrices from the SuiteSparse collection ([7]), analyzing strong scaling, load balance, communication volume and the influence of matrix structure on performance.

II. METHODOLOGY

In this distributed SpMV investigation we used our previous work ([1]) on single-GPU SpMV as a baseline, specifically a CSR (*Compressed Sparse Row*) kernel with warp-level parallelism and shared memory reduction, alongside a CPU reference implementation. SuiteSparse ([7]) matrices were selected to ensure **strong consistency** across experiments, allowing us to study the influence of matrix topology (*symmetry*, *bandwidth*, *sparsity pattern*) on multi-GPU performance.

Our distributed implementation relies on **OpenMPI 4.1.5** ([8]), An open-source Message Passing Interface, a library specification for message-passing, proposed as a standard to program distributed memory systems by assuming message-based semantics, with **CUDA-aware MPI** support (see III-A). To guide our implementation we followed *Foster’s Methodology* ([2]), a four-stage framework: Partitioning, Communication, Agglomeration and Mapping, that provides a structured path from problem definition to efficient parallel implementation. To clarify the visual representation we provided below a sample of the book written by Foster. I figure III:

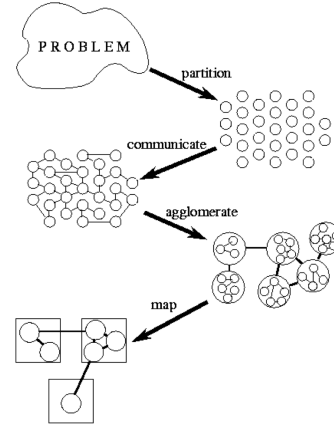


Fig. 1. PCAM: a design methodology for parallel programs)

All along this paper, we will denote our SpMV implementation as $Ax = y$ where A is a sparse matrix of size M (rows) by N (columns) with real coefficients (Float type in our implementation) with a certain amount of non zeros datas (Nnz: number of Non-zeros coefficients), x is a vector of N elements. Finally y is the result of our SpMV implementation.

A **1D cyclic distribution** was adopted to partition the sparse matrix rows across MPI processes. Each rank p owns rows $\{i \mid i \bmod P = p\}$, where P is the total number of processes. This ensures a natural load balance when the non-zeros are uniformly distributed across rows.

The vector x is also distributed across ranks following the same cyclic rule. Firstly, to the local SpMV computation, each rank identifies the *ghost entries* remote x_j values required for its local rows but owned by other ranks. These are exchanged via `MPI_Alltoallv`, following a 1D SpMV communication pattern: Identify required remote column indices, exchange vector values with owner ranks, perform local SpMV on the GPU and finally gather distributed result via `MPI_Gatherv`. After the scatter phase, each rank converts its local COO representation into CSR format. Column indices are remapped to a local extended vector $\tilde{x} = [x_{\text{owned}} \mid x_{\text{ghost}}]$ of size $|x_{\text{local}}| + |x_{\text{ghost}}|$ (concatenation).

Performance is evaluated using **strong scaling**: the matrix size is fixed while the number of GPUs varies from 1 to 4. Each experiment runs **30 iterations** after **10 warmup iterations** to amortize initialization costs. Speedup and efficiency are defined as:

$$s_p \text{ (speedup)} = \frac{t_1}{t_p}, \quad e_p \text{ (efficiency)} = \frac{s_p}{P} \quad (1)$$

where t_1 is the execution time on a single GPU and t_p the mean execution time across P GPUs.

For the state-of-the-art context on distributed SpMV optimizations, we drew inspiration from Gao, Ji and Wang ([3]).

All experiments were conducted on **1 to 4 NVIDIA Ampere A30 GPUs**. Key hardware specifications are summarized in Table III.

TABLE I
NVIDIA A30 SPECIFICATIONS (Via Cuda Device Properties)

Memory clock rate	1215 Mhz
Memory Bus Width	3072 Mhz
Peak Memory bandwidth	933.120 GB/s
Multiprocessors	56
Max number of threads/multiprocessor	2048
Max number of threads/block	1024
Max dimension size of thread block (x,y,z)	(1024, 1024, 64)
Max dimension size of a grid size	(2147483647, 65535, 65535)
Total amount of shared memory per block	49152 bytes

To optimize communication and minimize host-device overhead, the implementation leverages **CUDA-aware MPI** directly during the vector exchange phase. Instead of relying on traditional CPU buffers, the data layout is orchestrated such that local processes communicate directly via device pointers.

A. CUDA-Aware MPI and Device Direct Communication

To enable efficient GPU-to-GPU communication, our implementation leverages **CUDA-aware MPI**, a feature of OpenMPI 4.1.5 that allows direct data transfers between GPU device memories without staging through the host CPU. This eliminates unnecessary host-device copies and reduces synchronization overhead.

Device Pointers in MPI Calls: All collective operations (MPI_Scatterv, MPI_Alltoallv, MPI_Gatherv) use **device pointers** (e.g., `d_localI`, `d_xExtended`) directly. Here, `d_localI` is a GPU device pointer, and the data is transferred directly to the GPU memory of the receiving rank.

Example of CUDA-Aware Usage: During the ghost exchange phase, the MPI_Alltoallv operation for vector values is performed directly on device memory: `MPI_Alltoallv(d_toSend, recvCounts.data(), rDispls.data(), MPI_FLOAT, d_xExtended + xLocalSize, sendCountsGhost.data(), sDispls.data(), MPI_FLOAT, MPI_COMM_WORLD)`. This ensures that the ghost values are written directly into the GPU memory of `d_xExtended`, avoiding any host-side buffering.

B. Ghost Entries Management

To handle non-local dependencies in the distributed SpMV computation, we implement a ghosting mechanism to fetch remote elements of the input vector x required for local computations. The process is divided into three key steps:

Identify Ghost Columns: For each local row i owned by rank p , we identify the columns j where $A_{i,j} \neq 0$ and j is not

owned by p (i.e., $\text{ROLE}(j, P) \neq p$). These columns correspond to ghost entries in the local computation.

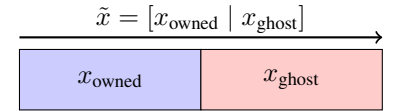
Exchange Ghost Indices: Each rank p sends the list of required ghost column indices to their respective owner ranks using MPI_Alltoallv. This step ensures that every rank knows which elements of x it needs to provide to others.

For each rank r , we build a list of columns `flatRequest` that are owned by r but required by the current rank. The size of these lists is stored in `sendCountsGhost[r]` and `recvCounts[r]`.

Exchange Ghost Values: After identifying the required ghost indices, we perform a second MPI_Alltoallv to exchange the actual values of x corresponding to these indices. The ghost values are placed in the extended local vector $\tilde{x} = [x_{\text{owned}} \mid x_{\text{ghost}}]$, where:

x_{owned} : Local portion of x owned by the rank (size: `xLocalSize`). x_{ghost} : Ghost values fetched from other ranks (size: `flatRequest.size()`).

The remapping of column indices to the extended vector is done using a hash map `globalToLocal`, which maps global column indices to their position in $\tilde{x}$.



Owned entries: Local portion of x

Ghost entries: Remote values fetched via MPI

Fig. 2. Structure of the extended vector $\tilde{x}$ combining owned and ghost entries for local SpMV computation.

C. Matrix Reading and Initial Distribution

The first step of our distributed SpMV implementation involves reading the sparse matrix from a file in **Matrix Market Exchange Format** using the `mmio` library. This step is performed exclusively by the root process (rank 0), which then distributes the matrix data to all other processes.

Matrix Reading: The root process reads the matrix file and extracts the dimensions (M, N) and the number of non-zero elements (`nnz`). The matrix is stored in **COO (Coordinate List)** format, with three arrays: `rows_array`: Global row indices of non-zero elements. `cols_array`: Global column indices of non-zero elements. `vals_array`: Non-zero values of the matrix.

Broadcasting Matrix Metadata: The root process broadcasts the matrix dimensions and `nnz` to all other processes using MPI_Bcast:

where `bufferMtxHeaders` contains $[M, N, \text{nnz}]$.

Partitioning Non-Zero Elements: The root process partitions the non-zero elements among all processes using a **1D cyclic distribution**. For each non-zero element at row i , its owner rank is determined by:

$$\text{owner} = \text{ROLE}(i, P) = i \bmod P \quad (2)$$

The root process then Counts the number of non-zero elements assigned to each rank (sendCounts). Computes the displacement for each rank (displs). Reorganizes the I, J, and valVect arrays so that all elements for rank p are contiguous.

Scattering Matrix Data: The root process scatters the partitioned I, J, and valVect arrays to all processes using MPI_Scatterv:

```
MPI_Scatterv(I.data(), ..., d_localI, ...)
MPI_Scatterv(J.data(), ..., d_localJ, ...)
MPI_Scatterv(valVect.data(), ... d_localVals, ...)
```

(3)

Here, d_localI, d_localJ, and d_localVals are **device pointers** allocated on each GPU.

D. Gathering the Result Vector y

After the local SpMV computations, the partial results stored in d_y on each GPU must be gathered to form the global result vector y . This is achieved using MPI_Gatherv, which accounts for the potentially uneven distribution of rows across processes.

Prepare Gather Parameters: Each process determines the number of rows it owns (numberLocalLine) and computes the displacements for the gather operation:

```
MPI_Allgather(&numberLocalLine, 1, MPI_INT,
gatherCounts.data(), 1, MPI_INT, MPI_COMM)
for (int r = 1; r < commSize; r++)
gatherDispls[r]=gatherDispls[r-1]
+ gatherCounts[r-1];
```

(4)

Allocate Global Result Vector: The root process allocates the global result vector d_globalY on its GPU:

```
cudaMalloc(&d_globalY, M * sizeof(DATATYPE))
```

(5)

Gather Partial Results: All processes contribute their local d_y to the global d_globalY using MPI_Gatherv:

```
MPI_Gatherv(d_y, numberLocalLine, MPI_FLOAT,
d_globalY, gatherCounts.data(),
gatherDispls.data(), MPI_FLOAT, 0, MPI_COMM)
```

(6)

This operation directly transfers the local results from each GPU to the root GPU's memory.

Reordering the Result Vector: Since the local rows are distributed cyclically, the gathered d_globalY is not in the original row order. The root process reorders the vector to match the original matrix row ordering:

```
for (int i = 0; i < M; i++) {
int owner = ROLE(i, commSize);
int pos = gatherDispls[owner] +
cursor[owner];
globalYOrdered[i] = globalY[pos];
cursor[owner]++; }
```

(7)

Here, globalYOrdered is the final result vector in the correct order.

E. Metrics and Load Balancing Metrics

In our work we track several architectural and performance metrics per rank to evaluate the efficiency of our 1D cyclic distribution: **Computational Load Balance:** We measure the number of non-zero elements (localNnz) allocated to each rank, monitoring the minimum, average, and maximum NNZ distribution to evaluate load balance across different matrix topologies. **Communication Volume:** Tracked via the size of the ghost cell arrays ($|x_{ghost}|$) and the size of the buffer exchanged during the MPI_Alltoallv step (totalToServe). **Throughput (GFLOPS):** Computed directly on the GPU using CUDA events to isolate the kernel execution time (t_p) from the MPI setup phase. For each rank p , the performance is defined by:

$$\text{GFLOPS}_p = \frac{2 \times \text{localNnz}}{t_{p,\text{mean}} \times 10^9} \quad (8)$$

where $t_{p,\text{mean}}$ represents the mean time in seconds over 30 measured iterations (excluding 10 warmup iterations) for this current rank. We choose 2 for the number of operations because in our CSR Warp we have 1 addition and 1 multiplication for each line. To get a global result we compute the GFLOPS by summing each GFLOPS of each rank. It is possible because here we used localNnz.

The global throughput of p ranks, when the ranks start at 1, is defined by :

$$\text{GFLOPS}_{\text{global}} = \sum_{i=1}^p \text{GFLOPS}_i \quad (9)$$

Global mean time: The global mean time in seconds (ms) of p ranks is defined by :

$$t_{\text{global}} = \max_{i \in [1,p]} t_{p,\text{mean}} \quad (10)$$

This is because the overall execution time is constrained by the slowest rank due to synchronization barriers (e.g., MPI_Barrier).

III. EXPERIMENTAL SETUP & REPRODUCIBILITY

A. Environnement

All experiments were conducted on a SLURM-managed (*Edu01*) cluster using the edu-short partition. Each node is equipped with up to 4 NVIDIA Ampere A30 GPUs (24 GB HBM2, compute capability 8.0). The software environment relies on **CUDA 12.5**, **OpenMPI 4.1.5** compiled with CUDA-aware support (`--with-cuda=internal`), and **GCC 12.3**.

B. Compilation

The code is compiled using `nvcc` with architecture flag `-arch=sm_80` and linked against `libmpi`. The full build is automated via a Makefile. You can see our repository with the different files and outputs on our **Github** : <https://github.com/steropes13/multi-gpu-spmv/tree/main>. The **file tree** is detailed in the README.md file, the main implementation is in `main.cu` file.

C. Execution

Jobs are submitted via SLURM **batch** scripts specifying `--ntasks` and `--gres=gpu` accordingly. CUDA-aware MPI is activated at runtime via `OMPI_MCA_opal_cuda_support=true`, and the matrix path is passed as a command-line argument to the binary. After building the project via the Makefile, which is invoked directly within the batch scripts, the execution command takes the following form:

```
mpirun --mca opal_cuda_support
1 -np <number_of_gpus> bin/main
<path-to-mtx-file>
```

In order to automate the submission of the 40 jobs corresponding to 1 to 4 GPUs across the 10 matrices of our dataset (see Table V-A), we provide a Bash script (available in `script/`) that generates all the required SLURM batch files. Each generated file follows the naming convention `sbatch-{matrix_name}-{num_processes}.sh` (e.g., `sbatch-cage15-4.sh`). By default if it is not necessary to get all the sbatch file `sbatch.sh` provided can be used and the values seen above (section IV-C) can be changed. Finally all the sbatch scripts we generate are with `ntasks=<number_of_gpus>`. In this `sbatch.sh` file, the `mpirun` line is commented because it is used only to execute the makefile with Cuda and MPCC loaded.

IV. EXPERIMENTAL RESULTS & PERFORMANCE ANALYSIS

In this section, we present the empirical performance gathered across 1, 2, 3 and 4 NVIDIA A30 GPUs. We analyze the behavior of our distributed SpMV implementation on both structured and unstructured matrices from the SuiteSparse collection ([7]).

A. Dataset

Table V-A summarizes the 10 matrices selected. These matrices were chosen to cover a variety of structural properties, symmetry, sparsity pattern, and problem size, in order to assess the behavior of our distributed SpMV implementation under different conditions.

TABLE II

DATASET OF 10 MATRICES TAKEN FROM SUITESPARSE ([7] WEBSITE FOR STRONG CONSISTENCY

	Rows	Cols	Nnz	Kind	Symmetry
cage15	5,154,859	5,154,859	99,199,551	Directed Weighted Graph	No
ASIC_680ks	682,712	682,712	1,693,767	Circuit Simulation Problem	No
vas_stokes_2M	2,146,677	2,146,677	65,129,037	Semiconductor Process Problem	No
poisson3Db	85,623	85,623	2,374,949	Computational Fluid Dynamics Problem	No
degme	185,501	659,415	8,127,528	Linear Programming Problem	No
1138_bus	1,138	1,138	4,054	Power Network Problem	Yes
shipsec5	179,86	179,86	4,598,604	Structural Problem	Yes
msdoor	415,863	415,863	19,173,163	Structural Problem	Yes
Ge87H76	112,985	112,985	7,892,195	Quantum Chemistry Problem	Yes
bmw3_2	227,362	227,362	11,288,630	Structural Problem	Yes

B. Strong Scaling and performance analysis

We analyse here our measurements on the local Kernel CSR Warp (with shared memory) SpMV

C. Strong Scaling and Performance Analysis

Figures V-C, ??, ?? and ?? respectively present the aggregate GFLOPS, mean iteration time, speedup and parallel efficiency across all tested configurations. The speedup is computed as $S_p = T_1/T_p$, where T_1 is the single-GPU baseline from [1] and T_p is the maximum mean iteration time across all p ranks, the bottleneck rank drives the overall performance.

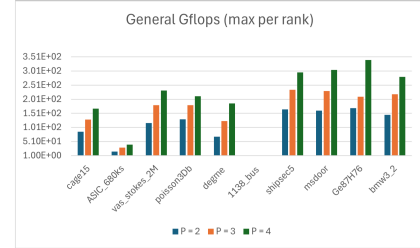


Fig. 3. GFLOPS_{global} of the 10 matrices across 2-4 GPUs for SpMV kernel with CSR Warp with shared memory

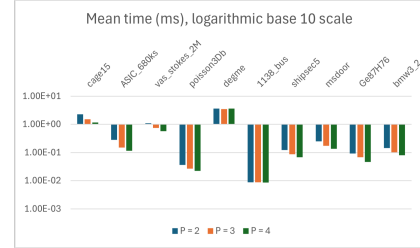


Fig. 4. t_{global} of the 10 matrices across 2-4 GPUs for SpMV kernel with CSR Warp with shared memory

1) *Throughput and Scaling Behavior:* The aggregate GFLOPS results demonstrate a consistent and near-linear growth with the number of GPUs for the majority of the tested matrices. cage15, the largest matrix in our dataset with approximately 99 million non-zeros, reaches 86 GFLOPS with 2 GPUs and scales up to 168 GFLOPS with 4 GPUs, reflecting a throughput increase close to $\times 2$ at each doubling of the GPU count. Similar behavior is observed for structured matrices such as msdoor, Ge87H76 and bmw3_2, which all exhibit regular sparsity patterns and benefit from good cache locality on the local CSR computation. vas_stokes_2M and shipsec5 follow the same trend, reaching respectively 232 and 296 GFLOPS at $P = 4$.

The exception is degme, whose mean iteration time remains nearly constant across all configurations (3.60, 3.50 and 3.66 ms for $P = 2, 3, 4$), resulting in a speedup close to 1.0 regardless of the number of GPUs. This behavior suggests that degme is dominated by an irregularity that prevents effective parallelization under our 1D cyclic distribution, likely a highly skewed non-zero distribution across rows, causing severe load imbalance at the kernel level despite the cyclic partitioning strategy. 1138_bus exhibits a similar pattern, with mean times around 8.8 ms for all configurations and a speedup oscillating

around 1.0, which is consistent with its very small size (only 4,720 non-zeros) making the MPI communication overhead dominate the actual computation time.

TABLE III

SPEEDUP HEAT-MAP OF THE 10 MATRICES ACROSS 2-4 GPUS FOR SpMV KERNEL WITH CSR WARP WITH SHARED MEMORY

	P = 2	P = 3	P = 4
cage15	2.35	3.51	4.61
ASIC_680ks	2.41	4.55	5.93
vas_stokes_2M	2.46	3.64	4.74
poisson3Db	3.06	4.14	4.97
degme	1.02	1.05	1.00
1138_bus	0.99	0.99	1.01
shipsec5	2.17	3.06	3.90
msdoor	2.22	3.19	4.11
Ge87H76	2.27	3.09	4.55
bmw3_2	2.19	3.12	3.96

2) *Speedup Analysis*: The speedup heatmap reveals super-linear speedup for several matrices, most notably ASIC_680ks (5.93 at $P = 4$), poisson3Db (4.97) and cage15 (4.61). Super-linear speedup, where $S_p > p$, arises when distributing the matrix across multiple GPUs allows each rank's local working set to fit more effectively into the GPU cache hierarchy, reducing the effective memory access latency compared to the single-GPU case where the full matrix and vector must be accessed from global memory. This effect is particularly pronounced for matrices with moderate non-zero counts per row and regular structure, where the local vector $\tilde{x} = [x_{\text{owned}} \mid x_{\text{ghost}}]$ is small enough to remain in L2 cache.

TABLE IV

EFFICIENCY HEAT-MAP OF THE 10 MATRICES ACROSS 2-4 GPUS FOR SpMV KERNEL WITH CSR WARP WITH SHARED MEMORY

	P = 2	P = 3	P = 4
cage15	1.17	1.17	1.15
ASIC_680ks	1.21	1.52	1.48
vas_stokes_2M	1.23	1.21	1.18
poisson3Db	1.53	1.38	1.24
degme	0.51	0.35	0.25
1138_bus	0.50	0.33	0.25
shipsec5	1.09	1.02	0.97
msdoor	1.11	1.06	1.03
Ge87H76	1.14	1.03	1.14
bmw3_2	1.09	1.04	0.99

3) *Parallel Efficiency*: The efficiency values consistently exceed 1.0 for most matrices, which is a direct consequence of the super-linear speedup discussed above and corroborates the cache effect hypothesis. Values above 1.15 are observed for cage15, vas_stokes_2M and ASIC_680ks, indicating that the distributed execution is more efficient per GPU than the sequential baseline. Conversely, degme and 1138_bus show efficiencies around 0.25 to 0.50, confirming that these matrices do not benefit from the parallelization strategy employed here.

4) *Impact of Ghost Entry Communication*: The ghost entry exchange, performed via CUDA-aware `MPI_Alltoallv` directly between device pointers, represents a non-negligible fraction of the total execution time for large matrices. For cage15 at $P = 4$, the ghost exchange accounts for several

milliseconds of setup overhead, while the SpMV kernel itself runs in approximately 1.16 ms per iteration. This overhead is amortized over the 30 measured iterations and is excluded from the reported mean time, which reflects kernel-only performance. In a real iterative solver scenario where the ghost exchange would be repeated at every iteration, the effective throughput would be lower, and communication-computation overlap via CUDA streams would become necessary.

It is worth noting that the CUDA-aware MPI support depends on the runtime activation of the `opal_cuda_support` flag, which requires the OpenMPI module to be loaded with the correct CUDA environment. In certain cluster configurations, a mismatch between the loaded CUDA module version and the one used to compile OpenMPI may silently fall back to host-staged transfers, partially undermining the CUDA-aware benefit without raising an explicit error. This could explain marginal inconsistencies observed in the ghost exchange timings across some runs.

D. Load Balance, Communication Volume and Matrix Structure

The 1D cyclic row distribution achieves near-perfect load balance across all configurations. For cage15 at $P = 4$, the NNZ spread across ranks stays below 0.6% (24,726,258 vs 24,872,948), and msdoor exhibits an even tighter imbalance of 0.16%. Only 1138_bus, with its 4,054 total non-zeros, reaches 5.6% — an expected artifact of its indivisible small size.

The ghost exchange time (`ghost_ms`) reveals a sharp contrast between structured and unstructured matrices. For cage15, the `MPI_Alltoallv` consistently takes ~4,700 ms regardless of P , far exceeding the kernel time of 1–4 ms per iteration, due to the irregular non-zero pattern requiring each rank to fetch entries from all other ranks. Conversely, msdoor stabilizes around 17–25 ms, reflecting the localized access pattern of structured finite element matrices. The initialization scatter consistently takes ~14,500 ms for cage15, highlighting that the current single-node matrix loading strategy would require distributed I/O to scale further.

The 1D distribution performs best on large, uniformly distributed matrices such as cage15 and msdoor, where the kernel fully occupies each GPU. It reaches its limits on very small matrices like 1138_bus, where MPI overhead dominates, and on highly irregular matrices where communication volume becomes the bottleneck.

E. Weak Scaling Analysis

To complement the strong scaling study, a weak scaling experiment was conducted by generating synthetic sparse matrices in Matrix Market format via a dedicated Python script. The script produces a square matrix of dimension $M = N = P \times 200,000$ with exactly 32 non-zeros per row, chosen to match the warp size of the NVIDIA A30 and ensure full warp utilization in the CSR warp kernel. The total problem size therefore scales proportionally with P , keeping the workload per GPU constant at 200,000 rows and 6,400,000 non-zeros. This setup isolates the communication overhead

from the computation scaling, as an ideal implementation would exhibit a constant execution time regardless of P .

The results indicate that the mean iteration time remains approximately stable across $P = 1$ to $P = 4$, increasing from ~ 1.2 ms at $P = 1$ to ~ 1.5 ms at $P = 4$ — a degradation of roughly 25%. This modest increase is attributable to the ghost entry exchange, whose volume grows with P as each rank must fetch a larger fraction of the distributed vector x from its neighbors. The total aggregate GFLOPS scales nearly linearly from ~ 43 GFLOPS at $P = 1$ to ~ 160 GFLOPS at $P = 4$, confirming that the per-GPU computational efficiency is well preserved. The synthetic matrix, with its perfectly uniform structure of 32 non-zeros per row, produces an ideal load balance (NNZ imbalance below 0.01%) and minimizes cache pressure on the local vector, explaining the near-constant kernel time. These results validate that our 1D cyclic distribution scales gracefully in the weak sense for regular, warp-aligned workloads, and that the MPI communication overhead remains a secondary cost for sufficiently large per-GPU problem sizes.

V. CONCLUSION

This work extended our previous single-GPU CSR warp SpMV kernel ([1]) to a multi-GPU distributed implementation using CUDA-aware MPI with a 1D cyclic row distribution. Experiments on 10 SuiteSparse matrices with up to 4 NVIDIA A30 GPUs showed near-linear speedup for large structured matrices such as cage15 and msdoor, with aggregate GFLOPS scaling from 44 to 168 for cage15 between $P = 1$ and $P = 4$. Small matrices like 1138_bus showed no benefit from parallelization, as MPI overhead dominates the computation. The weak scaling study confirmed stable per-GPU efficiency for regular workloads.

The main limitation is the initialization phase, where rank 0 loads the full matrix before scattering it, bounding the problem size to the CPU memory of a single node. Future work (as [3]) could address this with distributed I/O, 2D partitioning to reduce ghost communication volume, and communication-computation overlap via CUDA streams.

VI. CONCLUSION

VII. ANNEXE

All the local results of each rank can be seen in the `.o` files generated each job via the script you can find in our repository.

REFERENCES

- [1] Titien MARIE, Previous work on SpMV investigation (in `report/`), <https://github.com/steropes13/gpu-sparse/>, May 2026
- [2] I. Foster. Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering. Parallel programming / scientific computing. ADDISON WESLEY Publishing Company Incorporated, 1995.
- [3] Gao, J. , Ji, W. and Wang, Y., Optimization of Large-Scale Sparse Matrix-Vector Multiplication on Multi-GPU Systems, November 2024
- [4] "Nvidia Cuspars documentation," docs.nvidia.com/cuda/cuspars/, May 2026.
- [5] "Github of Nvidia Cuspars implementation", github.com/NVIDIA/CUDALibrarySamples/tree/main/cuSPARSE, May 2026.
- [6] CUDA programming guide, May 2026
- [7] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. ACM Transactions on Mathematical Software 38, 1, Article 1 (December 2011), 25 pages. DOI: <https://doi.org/10.1145/2049662.2049663>, SuiteSparse matrices website.
- [8] Open-MPI documentation, <https://www.open-mpi.org> June 2026